

Лабораторная работа № 2
Криптоанализ аффинного шифра
Вариант №15

Ф. И. О. студента: Рау Алексей Евгеньевич

Группа: ФИТ-231

Проверил: Белим С.Ю.

Дата: 11.11.2025

Основные сведения

Формула для зашифрования текста:

$$y_i = E_k(x_i) = (ax_i + b) \bmod m$$

Формула для расшифрования текста:

$$x_i = a^{-1}(y_i - b) \bmod m$$

Результаты

ШИФР-ТЕКСТ (ШТ):

ыдеыжуовыушуйуивйошяжыаоадпщйаспсцсфежнцухвяадйвеэндсицуы
ыутсяыубскцсбтойщнцукжкотяэпубсыуиуцнуйошдбдадыэоыжшуяиотндйэн
сяищуууяжыукжшыдяидйвб сянщупуцубтойщчкйэтуыэ

Результат частотного анализа ШТ:

буква	а	б	в	г	д	е/ё	ж	з	и	й
частота	5	6	5	0	10	3	7	0	7	11
буква	к	л	м	н	о	п	р	с	т	у
частота	5	0	0	6	9	4	0	10	6	20
буква	ф	х	ц	ч	ш	щ	ъ	ы	ь	э
частота	1	1	7	2	5	5	0	12	0	6
буква	ю	я								
частота	1	9								

Наиболее часто встречающиеся символы: у, ы, й, д, с

Системы уравнений:

$$\begin{cases} (14a + b) \bmod 32 \equiv 19 \\ (5a + b) \bmod 32 \equiv 4 \end{cases}$$

или

$$\begin{cases} (14a + b) \bmod 32 \equiv 4 \\ (5a + b) \bmod 32 \equiv 19 \end{cases}$$

Решения систем уравнений:

1. $a = 23, b = 17$

2. $a = 9, b = 6$

ВЕРНЫЙ КЛЮЧ: $a = 9, b = 6$

РАСШИФРОВАННЫЙ ТЕКСТ (ОТ):

но знаешь не велеть ливсанки кобылку бурую за пречьскользю поутренне мусне гуд
руг милый предадимся бегу нетерпеливого коня и навестим поля пустые лесане дав
но столько густые и берег милый для меня

Код программы

```
import tkinter as tk
from tkinter import filedialog
import matplotlib
matplotlib.use('TkAgg')

russian_alphabet = 'абвгдежзийклмнопрстуфхцчщъыьэюя'
russian_alphabet_lenght = len(russian_alphabet)
frequent_letters = 'оеаитнсрвлкмдпужызьбгчйхжюшцщэф'

def gcd_and_bezout(a, b):
    if b == 0:
        return abs(a), (1 if a > 0 else -1), 0

    gcd_val, x1, y1 = gcd_and_bezout(b, a % b)
    x = y1
    y = x1 - (a // b) * y1

    return gcd_val, x, y

def mod_inverse(a, m):
    gcd_val, x, y = gcd_and_bezout(a, m)
    if gcd_val != 1:
        return None
    return x % m

def solve_linear_congruence(a, b, m):
    gcd, x0, y0 = gcd_and_bezout(a, m)

    if b % gcd != 0:
        return []

    x = (x0 * (b // gcd)) % m
```

```

m_div_gcd = m // gcd
solutions = []
for i in range(gcd):
    solution = (x + i * m_div_gcd) % m
    solutions.append(solution)

solutions.sort()

return solutions

def solve_system_of_congruences(a, c, b, d, m):
    diff_a = (a - c) % m
    diff_b = (b - d) % m

    x_solutions = solve_linear_congruence(diff_a, diff_b, m)

    if not x_solutions:
        return []

    solutions = []
    for x in x_solutions:
        y = (b - a * x) % m

        if (c * x + y) % m == d % m:
            solutions.append((x, y))
        else:
            y = (d - c * x) % m
            if (a * x + y) % m == b % m:
                solutions.append((x, y))

    return solutions

def get_text_input():
    print("\n"+"="*50)
    print("Выберите способ ввода текста:")
    print("1. Ввод с консоли")
    print("2. Чтение из файла")

    choice = input("\nВаш выбор (1-2): ").strip()

    if choice == '1':
        print("\n"+"="*50)
        text = input("Введите текст: ")
        cleaned_text = ''.join([char for char in text if char in
russian_alphabet])
        print(f"Текст после очистки: {cleaned_text}")
        print(f"Длина очищенного текста: {len(cleaned_text)} символов")
        return cleaned_text
    elif choice == '2':
        text = read_from_file_dialog()
        if text:
            cleaned_text = ''.join([char for char in text if char in
russian_alphabet])
            print(f"Текст после очистки: {cleaned_text}")
            print(f"Длина очищенного текста: {len(cleaned_text)} символов")
            return cleaned_text
        return None
    else:
        print("Неверный выбор!")
        return None

```

```

def read_from_file_dialog():
    try:
        root = tk.Tk()
        root.withdraw()
        root.attributes('-topmost', True)

        file_path = filedialog.askopenfilename(
            title="Выберите текстовый файл",
            filetypes=[("Текстовые файлы", "*.txt"), ("Все файлы", "*.*")]
        )

        root.destroy()

        if not file_path:
            print("Выбор файла отменен.")
            return None

        print(f"Выбран файл: {file_path}")

        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read().strip()
            print(f"Файл успешно прочитан. Размер: {len(content)} символов")
            return content

    except Exception as e:
        print(f"Ошибка при чтении файла: {e}")
        return None

def frequency_analysis(text):

    if not text:
        print("Текст пустой!")
        return {}

    cleaned_text = text.lower()

    cleaned_text = cleaned_text.replace('ё', 'е')
    cleaned_text = cleaned_text.replace('ь', 'б')

    russian_chars = [char for char in cleaned_text if char in
russian_alphabet]
    cleaned_text = ''.join(russian_chars)

    total_chars = len(cleaned_text)

    if total_chars == 0:
        print("В тексте нет русских букв!")
        return {}

    freq_dict = {}
    for char in cleaned_text:
        if char in freq_dict:
            freq_dict[char] += 1
        else:
            freq_dict[char] = 1

    for char in freq_dict:
        freq_dict[char] = freq_dict[char] / total_chars

    sorted_freq = dict(sorted(freq_dict.items(), key=lambda item: item[1],
reverse=True))

```

```

        return sorted_freq

def symbol_to_num(symbol):
    return russian_alphabet.index(symbol)

def num_to_symbol(num):
    return russian_alphabet[num % russian_alphabet_lenght]

def decryption(text, a, b):
    result = []
    a_inverse = mod_inverse(a, russian_alphabet_lenght)
    if a_inverse == None:
        return None
    for char in text:
        x = symbol_to_num(char)
        y = a_inverse * (x - b) % russian_alphabet_lenght
        result.append(num_to_symbol(y))
    return ''.join(result)

def hypothesis_check(first_char, second_char, text, log_file):
    first_num = symbol_to_num(first_char)
    second_num = symbol_to_num(second_char)
    first_alphabet_num = symbol_to_num(frequent_letters[0])
    second_alphabet_num = symbol_to_num(frequent_letters[1])

    log_file.write(f"\nПроверяем гипотезу: '{first_char}' -> 'o',
'{'second_char}' -> 'e'\n")
    log_file.write(f"Уравнения:\n")
    log_file.write(f"    {first_alphabet_num}*a + b ≡ {first_num} (mod
{russian_alphabet_lenght})\n")
    log_file.write(f"    {second_alphabet_num}*a + b ≡ {second_num} (mod
{russian_alphabet_lenght})\n")

    solutions = solve_system_of_congruences(
        first_alphabet_num, second_alphabet_num,
        first_num, second_num,
        russian_alphabet_lenght
    )

    if not solutions:
        log_file.write(f"Система не разрешима\n")
        print(f"    Гипотеза '{first_char}'->o, {'second_char'}->e' не дала
решений")
        return []

    log_file.write(f"Найдено {len(solutions)} решений системы:\n")

    valid_solutions = []
    for i, (a, b) in enumerate(solutions, 1):
        a_inverse = mod_inverse(a, russian_alphabet_lenght)
        if a_inverse is None:
            log_file.write(f"    Решение {i}: a = {a}, b = {b} - не подходит (a
не взаимно прост с {russian_alphabet_lenght})\n")
            continue

        final_text = decryption(text, a, b)
        log_file.write(f"    Решение {i}: a = {a}, b = {b}\n")
        log_file.write(f"        Расшифровка (первые 100 символов):
{final_text[:100]}...\n")

        valid_solutions.append((a, b, final_text))

```

```

        return valid_solutions

def crack_affine_cipher(text):
    log_filename = "decryption_protocol.txt"
    with open(log_filename, 'w', encoding='utf-8') as log_file:
        log_file.write("="*60 + "\n")
        log_file.write("ПРОТОКОЛ РАСШИФРОВКИ АФФИННОГО ШИФРА\n")
        log_file.write("="*60 + "\n\n")
        log_file.write(f"Исходный зашифрованный текст: {text[:100]}...\n")
        log_file.write(f"Длина текста: {len(text)} символов\n\n")

        sorted_freq = frequency_analysis(text)
        log_file.write("Частотный анализ зашифрованного текста:\n")
        for char, freq in sorted_freq.items():
            log_file.write(f"  '{char}': {freq:.5f}\n")
        log_file.write("\n")

        frequent_chars = list(sorted_freq.keys())

        log_file.write(f"Начинаем перебор гипотез для {len(frequent_chars)}
наиболее частых символов...\n")
        log_file.write("-"*60 + "\n")

        print(f"Начинаем перебор гипотез... (протокол записывается в
{log_filename})")

        for i in range(len(frequent_chars)):
            for j in range(i+1, len(frequent_chars)):
                first_char = frequent_chars[i]
                second_char = frequent_chars[j]

                log_file.write(f"\n{'='*40}\n")
                log_file.write(f"Гипотеза {i*len(frequent_chars)+j+1}:
'{first_char}' -> 'o', '{second_char}' -> 'e'\n")

                results = []

                solutions1 = hypothesis_check(first_char, second_char, text,
log_file)
                results.extend(solutions1)

                log_file.write(f"\nПроверяем обратный порядок: '{second_char}'
-> 'o', '{first_char}' -> 'e'\n")
                solutions2 = hypothesis_check(second_char, first_char, text,
log_file)
                results.extend(solutions2)

                if results:
                    log_file.write(f"\n{'='*60}\n")
                    log_file.write(f"НАЙДЕНО {len(results)} ВАРИАНТОВ
КЛЮЧЕЙ!\n")

                    print(f"\nНайдено {len(results)} вариантов ключей для
гипотезы {first_char}/{second_char}:")

                    for idx, (a, b, decrypted) in enumerate(results, 1):
                        log_file.write(f"\nВариант {idx}:\n")
                        log_file.write(f"  Ключ: a = {a}, b = {b}\n")
                        log_file.write(f"  Расшифрованный текст:
{decrypted[:150]}...\n")

                    print(f"\nВариант {idx}: a={a}, b={b}")

```

```

        print(f"Текст: {decrypted[:100]}...")

        choice = input(f"\nЭто осмысленный текст? (y - да, n -
нет, s - сохранить и продолжить): ").lower()
        log_file.write(f" Реакция пользователя: {choice}\n")

        if choice == 'y':
            log_file.write(f"\n{' '*60}\n")
            log_file.write(f"ПОЛЬЗОВАТЕЛЬ ПОДТВЕРДИЛ

ОСМЫСЛЕННОСТЬ ТЕКСТА!\n")

            log_file.write(f"ФИНАЛЬНЫЙ КЛЮЧ: a = {a}, b =

{b}\n")

            log_file.write(f"ПОЛНЫЙ ТЕКСТ:\n{decrypted}\n")
            log_file.write(f"{' '*60} + "\n")

            print(f"\nПротокол сохранен в {log_filename}")
            return a, b, decrypted

        elif choice == 's':
            save_filename = f"decrypted_variant_{idx}.txt"
            with open(save_filename, 'w', encoding='utf-8') as
f:

                f.write(f"Ключ: a={a}, b={b}\n\n")
                f.write(decrypted)
                print(f"Сохранено в {save_filename}")
                log_file.write(f" Текст сохранен в файл:

{save_filename}\n")

            log_file.write(f"\n" + "{' '*60} + "\n")
            log_file.write(f"ПЕРЕБОР ЗАВЕРШЕН. ПОДХОДЯЩИЙ КЛЮЧ НЕ НАЙДЕН.\n")
            log_file.write(f"{' '*60} + "\n")
            print(f"\nПеребор завершен. Подходящий ключ не найден.")
            print(f"Протокол сохранен в файл: {log_filename}")

        return None, None, None

def modular_arithmetic_menu():
    """Меню для проверки модульной арифметики"""
    while True:
        print("\n" + "{' '*50}")
        print("МОДУЛЬНАЯ АРИФМЕТИКА")
        print("{' '*50}")
        print("1. Нахождение НОД и коэффициентов Безу")
        print("2. Поиск обратного элемента")
        print("3. Решение линейного сравнения  $ax \equiv b \pmod{m}$ ")
        print("4. Решение системы сравнений")
        print("5. Назад в главное меню")
        print("{' '*50}")

        choice = input("\nВаш выбор (1-5): ").strip()

        if choice == '1':
            print("\n--- НОД и коэффициенты Безу ---")
            try:
                a = int(input("Введите первое число (a): "))
                b = int(input("Введите второе число (b): "))

                gcd, x, y = gcd_and_bezout(a, b)

                print(f"\nРезультат:")
                print(f" НОД({a}, {b}) = {gcd}")
                print(f" Коэффициенты Безу: x = {x}, y = {y}")

```

```

except ValueError:
    print("Ошибка! Введите целые числа.")

elif choice == '2':
    print("\n--- Поиск обратного элемента ---")
    try:
        a = int(input("Введите число a: "))
        m = int(input("Введите модуль m: "))

        inverse = mod_inverse(a, m)

        if inverse is None:
            print(f"\nОбратный элемент для {a} по модулю {m} не
существует")
        else:
            print(f"\nОбратный элемент для {a} по модулю {m}:
{inverse}")

except ValueError:
    print("Ошибка! Введите целые числа.")

elif choice == '3':
    print("\n--- Решение линейного сравнения ---")
    try:
        a = int(input("Введите коэффициент a: "))
        b = int(input("Введите правую часть b: "))
        m = int(input("Введите модуль m: "))

        solutions = solve_linear_congruence(a, b, m)

        print(f"\nСравнение: {a}x ≡ {b} (mod {m})")

        if not solutions:
            print("Решений нет")
            gcd = gcd_and_bezout(a, m)[0]
            print(f"Причина: НОД({a}, {m}) = {gcd} не делит {b}")
        else:
            print(f"Все решения: {'', '.join(str(x) for x in
solutions))}")

except ValueError:
    print("Ошибка! Введите целые числа.")

elif choice == '4':
    print("\n--- Решение системы сравнений ---")
    print("Система вида:")
    print("  (a*x + y) ≡ b (mod m)")
    print("  (c*x + y) ≡ d (mod m)")
    print("-"*40)

    try:
        a = int(input("Введите коэффициент a (из первого уравнения):
"))
        c = int(input("Введите коэффициент c (из второго уравнения):
"))

        b = int(input("Введите b (правая часть первого уравнения): "))
        d = int(input("Введите d (правая часть второго уравнения): "))
        m = int(input("Введите модуль m: "))

        solutions = solve_system_of_congruences(a, c, b, d, m)

```



```

        print(f"\nСистема:")
        print(f"  ({a})x + y ≡ {b} (mod {m})")
        print(f"  ({c})x + y ≡ {d} (mod {m})")
        print("-"*40)

        if not solutions:
            print("Решений нет")
        else:
            print(f"Найдено решений: {len(solutions)}")
            print("\nВсе решения (x, y):")

            for i, (x, y) in enumerate(solutions, 1):
                print(f"  {i:2d}. x = {x}, y = {y}")

    except ValueError:
        print("Ошибка! Введите целые числа.")

elif choice == '5':
    print("Возврат в главное меню...")
    break

else:
    print("Неверный выбор! Попробуйте снова.")

def cypher_menu():
    while True:
        print("\n" + "="*50)
        print("АФИННЫЙ ШИФР")
        print("="*50)
        print("1. Частотный анализ")
        print("2. Взлом шифра")
        print("3. Назад в главное меню")
        print("="*50)

        choice = input("\nВаш выбор (1-3): ").strip()

        if choice == '1':
            text = get_text_input()

            if text:
                print(f"\n" + "="*50)
                print("РЕЗУЛЬТАТЫ ЧАСТОТНОГО АНАЛИЗА")
                print("="*50)

                frequencies = frequency_analysis(text)
                if frequencies:
                    print("\nСимвол | Частота    | Количество")
                    print("-"*35)
                    for char, freq in frequencies.items():
                        count = int(freq *
len(text.replace('ë', 'e').replace('ъ', 'ь').lower()))
                        print(f"{char:^7} | {freq:.5f}    | {count}")
                elif choice == '2':
                    text = get_text_input()
                    if text:
                        crack_affine_cipher(text)
                elif choice == '3':
                    print("Возврат в главное меню...")
                    break
            else:
                print("Неверный выбор! Попробуйте снова.")

```

```
def main():

    while True:
        print("\n"+"="*50)
        print("ВЫБЕРИТЕ ДЕЙСТВИЕ:")
        print("1. Проверка модульной арифметики")
        print("2. Взлом варианта")

        choice = input("\nВаш выбор (1-2): ").strip()

        if choice == '1':
            modular_arithmetic_menu()
        elif choice == '2':
            cypher_menu()

if __name__ == "__main__":
    main()
```